版本: **1.0.4　2024 年 03 月**

AnyCloud39AV100 NNE API 参考

© 2024 Guangzhou Anyka Microelectronics Co., Ltd.　All rights reserved

声　　明

本手册的版权归广州安凯微电子股份有限公司所有，受相关法律法规的保护。未经广州安凯微电子股份有限公司的事先书面许可，任何人不得复制、传播本手册的内容。

本手册所涉及的知识产权归属广州安凯微电子股份有限公司所有（或经合作商授权许可使用），任何人不得侵犯。

本手册不对包括但不限于下列事项担保：适销性、特殊用途的适用性；实施该用途不会侵害第三方的知识产权等权利。

广州安凯微电子股份有限公司不对由使用本手册或执行本手册内容而带来的任何损害负责。

本手册是按当前的状态提供参考，随附产品或本书内容如有更改，恕不另行通知。

联　系　方　式

广州安凯微电子股份有限公司

地址：广州市黄埔区博文路 107 号安凯微电子 H 大厦

电话: (86)-20-3221 9000

传真: (86)-20-3221 9258

邮编: 510555

销售热线:

(86)-20-3221 9499

电子邮箱:

sales@anyka.com

主页:

http: //www. Anyka.com

Anyka Confidential For Giec Use Only

版 本 变 更 说 明

以下表格对于本文档的版本变更做一个简要的说明。版本变更仅限于技术内容的变更，不包括版式、格式、句法等的变更。

版本	说明	完成日期
V1.0.0	正式发布	2022 年 11 月
V1.0.1	修改 NNE_Run API	2022 年 12 月
V1.0.2	增加 NNE_Create_CH_From_Model_Buffer	2023 年 04 月
V1.0.3	接口分为两部分：libplat_npu 和 libak_nne。	2023 年 11 月
V1.0.4	补充接口说明	2024 年 03 月

Anyka Confidential For Giec Use Only

目录

1 libplat_npu 5
1.1 概述 5
1.2 API 参考 5
1.2.1 ak_npu_get_version 5
1.2.2 ak_npu_init 5
1.2.3 ak_npu_deinit 6
1.2.4 ak_npu_reset_hw 7
1.3 错误码 7
2 libak_nne 7
2.1 概述 7
2.2 ak_nne_types.h 7
2.3 ak_nne_cbfun.h 8
2.3.1 T_AKCNN_DMA_BUFFER 8
2.3.2 AKCNN_CB_FUN_PRINTF 9
2.3.3 AKCNN_CB_FUN_MALLOC 9
2.3.4 AKCNN_CB_FUN_FREE 10
2.3.5 AKCNN_CB_FUN_DMA_ALLOC 10
2.3.6 AKCNN_CB_FUN_DMA_FREE 11
2.3.7 AKCNN_CB_FUN_BEGIN_CPU_ACCESS 12
2.3.8 AKCNN_CB_FUN_END_CPU_ACCESS 12
2.3.9 T_AKCNNLIB_COMMON_CB 13
2.4 ak_nne_common.h 14
2.4.1 NNE_INPUT_COLOR_E 15
2.4.2 NNE_INPUT_NORM_E 15
2.4.3 NNE_TASK_E 16
2.4.4 NNE_QUAN_METHOD_E 16
2.4.5 NNE_XRELU_TYPE_E 17
2.4.6 NNE_OP_TYPE_E 18
2.4.7 NNE_POOL_TYPE_E 19
2.4.8 NNE_DATA_TYPE_E 20
2.4.9 NNE_DATA_ARRANGE_E 20
2.4.10 NNE_MODEL_HEADER 21
2.4.11 NNE_INPUT_OUTPUT_BLOB_INFO_S 22
2.4.12 NNE_BLOB_VECTOR_S 22
2.4.13 NNE_POINT_S 23
2.4.14 NNE_RECT_S 23
2.4.15 NNE_Init 24
2.4.16 NNE_DeInit 24
2.4.17 NNE_Get_CallBackFun 25
2.4.18 NNE_Create_CH 25
2.4.19 NNE_Destroy_CH 26
2.4.20 NNE_Run 27

Anyka Confidential For Free Use Only

2.4.21 NNE_Run_Interrupt 29
2.4.22 NNE_Get_Lib_Version 30
2.5 错误码.....30
3 调用流程.....31

Anyka Confidential For Giec Use Only

1 libplat_npu

1.1 概述

初始化 npu 模块硬件资源，然后就可通过算法模型库实现各种不同的智能检测识别等功能。

1.2 API 参考

本模块提供以下 API 参考，如下表所示。

库接口	描述
ak_npu_get_version	获取 npu 库的版本号
ak_npu_init	初始化 npu 模块
ak_npu_deinit	销毁 npu 模块
ak_npu_reset_hw	重置 npu 硬件

1.2.1 ak_npu_get_version

【功能】

获取 npu 库的版本号。

【语法】

```
const char* ak_npu_get_version(void);
```

【参数】

参数名称	描述	取值范围	输入/输出
void	-	-	-

【返回值】

返回值	描述
npu 模块版本号	成功。
NULL	失败。

【需求】

头文件：ak_npu.h

库文件：libplat_npu.so 或者 libplat_npu.a

1.2.2 ak_npu_init

【功能】

初始化 npu 模块。

【语法】

```
int ak_npu_init(void);
```

【参数】

Anyka Confidential For Gic Use Only

参数名称	描述	取值范围	输入/输出
void	-	-	-

【返回值】

返回值	描述
0	成功。
非 0	失败，其值为错误码。

【需求】
头文件：ak_npu.h
库文件：libplat_npu.so 或者 libplat_npu.a

【注意】
在 libak_nne 库的接口被调用之前，必须先调用 ak_npu_init 接口。
函数不可重入，多次重入会报错。
配对函数：ak_npu_deinit()

1.2.3 ak_npu_deinit

【功能】
销毁 npu 模块。

【语法】

```
int ak_npu_deinit(void);
```

【参数】

参数名称	描述	取值范围	输入/输出
void	-	-	-

【返回值】

返回值	描述
0	成功。
非 0	失败，其值为错误码。

【需求】
头文件：ak_npu.h
库文件：libplat_npu.so 或者 libplat_npu.a

【注意】
与 ak_npu_init()配对使用。
在 ak_npu_deinit 之前，NNE 模块必须先销毁。
函数不可重入，多次重入会报错。
配对函数：ak_npu_init()

Anyka Confidential For Giec Use Only

1.2.4 ak_npu_reset_hw

【功能】

重置 npu 模块。

【语法】

```
int ak_npu_reset_hw(void);
```

【参数】

参数名称	描述	取值范围	输入/输出
void	-	-	-

【返回值】

返回值	描述
0	成功。
非 0	失败。

【需求】

头文件：ak_npu.h

库文件：libplat_npu.so 或者 libplat_npu.a

【注意】

当 NNE 模块运行出现错误时调用此接口。

1.3 错误码

错误码	含义
ERROR_NPU_ALREADY_INIT	NPU 模块已经初始化
ERROR_NPU_ALREADY_DEINIT	NPU 模块已经销毁

Anyka Confidential For Giec Use Only

2 libak_nne

2.1 概述

介绍 libak_nne 库的头文件信息。

2.2 ak_nne_types.h

该头文件内定义数据类型别称。

【定义】

```
typedef    unsigned char        T_U8;
typedef    signed char          T_S8;
typedef    unsigned short       T_U16;
typedef    signed short         T_S16;
```

```
typedef   unsigned long          T_U32;
typedef   signed long            T_S32;
typedef   unsigned long long     T_U64;
typedef   signed long long       T_S64;
typedef   T_S8 *                 T_pSTR;
typedef   const T_S8 *           T_pCSTR;
typedef   void                   T_VOID;
typedef   T_VOID *               T_pVOID;
typedef   T_U8                   T_BOOL;
#define   AK_FALSE               0
#define   AK_TRUE                1
#define   AK_NULL                0//((T_VOID*)0)
```

2.3 ak_nne_cbfun.h

该头文件内定义回调函数别称和 DMA 内存信息结构体。用户在使用相关功能时请使用回调函数而非 C 库函数。

2.3.1 T_AKCNN_DMA_BUFFER

【定义】

```
typedef struct T_AKCNN_DMA_BUFFER
{
    T_U8      *pData;   // virtual address of the buffer
    T_U32      pa;      // physical address of the buffer
    T_U32      size;    // size of the buffer
    T_S32      fd;

}T_AKCNN_DMA_BUFFER;
```

【成员】

成员名称	描述
pData	该 buffer 的虚拟地址
pa	该 buffer 的物理地址
size	该 buffer 的大小(byte)
fd	刷 cache 一致性时使用到

【示例】

```
//申请
T_AKCNN_DMA_BUFFER  *buf   =   (T_AKCNN_DMA_BUFFER
*)cb_fun.ak_FunDmaAlloc(size);
//打印
cb_fun.ak_FunPrintf((T_pCSTR)("virtual_addr=0x%x\n"), buf->pData);
```

Anyka Confidential For General Only

```
cb_fun.ak_FunPrintf((T_pCSTR)("physical_addr=0x%x\n"), buf->pa);
cb_fun.ak_FunPrintf((T_pCSTR)("size=%d\n"), buf->size);
//释放
cb_fun.ak_FunDmaFree(buf);
buf = NULL;
```

2.3.2 AKCNN_CB_FUN_PRINTF

【功能】

格式化输出，功能同 printf 函数。

【语法】

```
typedef    T_VOID    (*AKCNN_CB_FUN_PRINTF)(T_pCSTR  format, ...);
```

【参数】

参数名称	描述	取值范围	输入/输出
format	格式化字符串	-	-
...	参量表	-	-

【返回值】

无

【需求】

头文件：ak_nne_cbfun.h

库文件：libak_nne.so 或者 libak_nne.a

【举例】

```
cb_fun.ak_FunPrintf((T_pCSTR)("NNE_Run begin!\n"));
cb_fun.ak_FunPrintf((T_pCSTR)("num=%d\n"), num);
```

【注意】

无

2.3.3 AKCNN_CB_FUN_MALLOC

【功能】

申请内存。

【语法】

```
typedef    T_pVOID (*AKCNN_CB_FUN_MALLOC)(T_U32   size);
```

【参数】

参数名称	描述	取值范围	输入/输出
size	申请的 byte 数量	T_U32	输入

【返回值】

Anyka Confidential For Giec Use Only

返回值	描述
非 NULL	成功。
NULL	失败。

【需求】
头文件：ak_nne_cbfun.h
库文件：libak_nne.so 或者 libak_nne.a

【举例】

```
T_U8 *buf    = cb_fun.ak_FunMalloc(size);
```

【注意】
与 AKCNN_CB_FUN_FREE 成对使用。

2.3.4 AKCNN_CB_FUN_FREE

【功能】
释放内存。

【语法】

```
typedef    T_VOID   (*AKCNN_CB_FUN_FREE)(T_pVOID  mem);
```

【参数】

参数名称	描述	取值范围	输入/输出
mem	待释放的指针	void *	输入

【返回值】
无

【需求】
头文件：ak_nne_cbfun.h
库文件：libak_nne.so 或者 libak_nne.a

【举例】

```
cb_fun.ak_FunFree(p_data);
```

【注意】
与 AKCNN_CB_FUN_MALLOC 成对使用。

2.3.5 AKCNN_CB_FUN_DMA_ALLOC

【功能】
申请 DMA 内存。

【语法】

```
typedef    T_AKCNN_DMA_BUFFER*   (*AKCNN_CB_FUN_DMA_ALLOC)(T_U32   zSize);
```

Anyka Confidential For Giec Use Only

【参数】

参数名称	描述	取值范围	输入/输出
zSize	申请的内存大小	T_U32	输入

【返回值】

返回值	描述
非 NULL	成功。
NULL	失败。

【需求】

头文件：ak_nne_cbfun.h

【举例】

```
T_AKCNN_DMA_BUFFER  *buf = cb_fun.ak_FunDmaAlloc(size);
```

【注意】

与 AKCNN_CB_FUN_DMA_FREE 成对使用。

2.3.6 AKCNN_CB_FUN_DMA_FREE

【功能】

释放已申请的 DMA 内存。

【语法】

```
typedef   T_BOOL   (*AKCNN_CB_FUN_DMA_FREE)(T_AKCNN_DMA_BUFFER*  hBuf);
```

【参数】

参数名称	描述	取值范围	输入/输出
hBuf	指针	-	输入

【返回值】

返回值	描述
1	成功。
0	失败。输入 hBuf 为空。

【需求】

头文件：ak_nne_cbfun.h

库文件：libak_nne.so 或者 libak_nne.a

【示例】

```
cb_fun.ak_FunDmaFree(buf);
```

【注意】

与 AKCNN_CB_FUN_DMA_ALLOC 成对使用。

Anyka Confidential For Giec Use Only

2.3.7 AKCNN_CB_FUN_BEGIN_CPU_ACCESS

【功能】

在使用 CPU 处理 DMA 内存数据的前后添加 AKCNN_CB_FUN_BEGIN_CPU_ACCESS 和 AKCNN_CB_FUN_END_CPU_ACCESS，将缓存中的数据更新到 DMA 确保后续硬件从 DMA 中取到正确的值、刷新缓存确保后续 CPU 读取到 DMA 中的数据而非缓存中的数据。

【语法】

```
typedef T_S32 (*AKCNN_CB_FUN_BEGIN_CPU_ACCESS) (T_VOID *addr, T_S32 size, T_S32 fd);
```

【参数】

参数名称	描述	取值范围	输入/输出
addr	dma buf 的成员 pData	-	输入
size	dma buf 的成员 size	-	输入
fd	dma buf 的成员 fd	-	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【需求】

头文件：ak_nne_cbfun.h

库文件：libak_nne.so 或者 libak_nne.a

【示例】

```
cb_fun.ak_FunBeginCpuAccess(buffer->pData, buffer->size, buffer->fd);
```

【注意】

与 AKCNN_CB_FUN_END_CPU_ACCESS 成对使用。

2.3.8 AKCNN_CB_FUN_END_CPU_ACCESS

【功能】

在使用 CPU 处理 DMA 内存数据的前后添加 AKCNN_CB_FUN_BEGIN_CPU_ACCESS 和 AKCNN_CB_FUN_END_CPU_ACCESS，将缓存中的数据更新到 DMA 确保后续硬件从 DMA 中取到正确的值、刷新缓存确保后续 CPU 读取到 DMA 中的数据而非缓存中的数据。

【语法】

```
typedef T_S32 (*AKCNN_CB_FUN_END_CPU_ACCESS) (T_VOID *addr, T_S32 size, T_S32 fd);
```

【参数】

参数名称	描述	取值范围	输入/输出

Anyka Confidential For Giec Use Only

addr	dma buf 的成员 pData	-	输入
size	dma buf 的成员 size	-	输入
fd	dma buf 的成员 fd	-	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【需求】

头文件：ak_nne_cbfun.h

库文件：libak_nne.so 或者 libak_nne.a

【示例】

```
cb_fun.ak_FunEndCpuAccess(buffer->pData, buffer->size, buffer->fd);
```

【注意】

与 AKCNN_CB_FUN_BEGIN_CPU_ACCESS 成对使用。

2.3.9 T_AKCNNLIB_COMMON_CB

【说明】

定义回调函数汇总结构体。

【定义】

```
typedef struct
{
    AKCNN_CB_FUN_PRINTF                 ak_FunPrintf;
    AKCNN_CB_FUN_MALLOC                 ak_FunMalloc;
    AKCNN_CB_FUN_FREE                   ak_FunFree;
    AKCNN_CB_FUN_READREGISTER           ak_FunReadRegister;
    AKCNN_CB_FUN_WRITEREGISTER_ARRAY    ak_FunWriteRegisterArray;
    AKCNN_CB_FUN_DMA_ALLOC              ak_FunDmaAlloc;
    AKCNN_CB_FUN_DMA_FREE               ak_FunDmaFree;
    AKCNN_CB_FUN_FLUSH_DCACHE           ak_FunFlushDcache;
    AKCNN_CB_FUN_TIMER                  ak_FunTimer;
    AKCNN_CB_FUN_TIMER_FINISH           ak_FunTimerFinish;
    AKCNN_CB_FUN_BEGIN_CPU_ACCESS       ak_FunBeginCpuAccess;
    AKCNN_CB_FUN_END_CPU_ACCESS         ak_FunEndCpuAccess;
    AKCNN_CB_FUN_MUTEX_CREATE           ak_FunMutexCreate;
    AKCNN_CB_FUN_MUTEX_DELETE           ak_FunMutexDelete;
    AKCNN_CB_FUN_MUTEX_GET              ak_FunMutexGet;
    AKCNN_CB_FUN_MUTEX_RELEASE          ak_FunMutexRelease;
```

Confidential For Use Only

```
    AKCNN_CB_FUN_HW_PROCESS                    ak_FunHwProcess;
}T_AKCNNLIB_COMMON_CB;
```

【成员】

成员名称	描述
ak_FunPrintf	打印函数
ak_FunMalloc	申请普通内存
ak_FunFree	释放普通内存
ak_FunReadRegister	-
ak_FunWriteRegisterArray	-
ak_FunDmaAlloc	申请 DMA 内存
ak_FunDmaFree	释放 DMA 内存
ak_FunFlushDcache	-
ak_FunTimer	-
ak_FunTimerFinish	-
ak_FunBeginCpuAccess	DMA-cache 一致性处理初始标志
ak_FunEndCpuAccess	DMA-cache 一致性处理结束标志
ak_FunMutexCreate	-
ak_FunMutexDelete	-
ak_FunMutexGet	-
ak_FunMutexRelease	-
ak_FunHwProcess	-

【注意】

未描述成员是库内部使用，用户不需关注。

2.4 ak_nne_common.h

本模块提供网络相关枚举、结构体，以及 libak_nne 库函数接口。汇总如下：

枚举或结构体	描述
NNE_INPUT_COLOR_E	输入图片颜色空间，例如 BGR、RGB、GRAY
NNE_INPUT_NORM_E	输入图片归一化方式
NNE_TASK_E	算法任务类型
NNE_QUAN_METHOD_E	量化方式
NNE_XRELU_TYPE_E	Relu 系列激活函数类型
NNE_OP_TYPE_E	算子类型
NNE_POOL_TYPE_E	池化算子类别，最大值池化或均值池化
NNE_DATA_TYPE_E	输入数据类型
NNE_DATA_ARRANGE_E	数据排列方式
NNE_MODEL_HEADER	模型文件头信息
NNE_INPUT_OUTPUT_BLOB_INFO_S	网络输入或输出数据块信息
NNE_BLOB_VECTOR_S	网络输入或输出数据块向量

Anyka Confidential For Spec Use Only

NNE_POINT_S	“点”的表示方式
NNE_RECT_S	“矩形”的表示方式

函数接口	描述
NNE_Get_CallBackFun	获取公共 handle 中的回调函数指针。
NNE_Create_CH	初始化指定网络句柄。
NNE_Destroy_CH	销毁指定网络句柄。
NNE_Run	执行指定网络正向推理。
NNE_Get_Model_Header	获取网络模型文件头。
NNE_Get_Lib_Version	获取 NNE 库版本号。

2.4.1 NNE_INPUT_COLOR_E

【说明】

指训练时的网络输入数据的颜色空间。在 PC 端仿真步骤中，check 和 simulate 步骤主函数中会根据此信息做预处理，将 opencv 读入的数据转换为该信息指定的颜色空间，使得推理和训练保持一致。用户也可以自行修改预处理方式。

【定义】

```
typedef enum NNE_INPUT_COLOR_E
{
    NNE_INPUT_COLOR_BGR  = 0x0,
    NNE_INPUT_COLOR_RGB  = 0x1,
    NNE_INPUT_COLOR_YUV  = 0x2,
    NNE_INPUT_COLOR_GRAY  = 0x3

}NNE_INPUT_COLOR_E;
```

【成员】

成员名称	描述
NNE_INPUT_COLOR_BGR	BGR
NNE_INPUT_COLOR_RGB	RGB
NNE_INPUT_COLOR_YUV	YUV
NNE_INPUT_COLOR_GRAY	单通道灰度图

【注意】

模型转换过程中的配置文件中需填写。

2.4.2 NNE_INPUT_NORM_E

【说明】

指训练时归一化方式。工具会根据其枚举值做相应调整，使得推理和训练一致。

【定义】

```
typedef enum NNE_INPUT_NORM_E
{
    NNE_INPUT_NORM_ABS1  = 0x0,     //y=(x-128)/128;     //x:[0,255],y:[-1,1]
    NNE_INPUT_NORM_01  = 0x1,       //y=(x-0)/255;       //x:[0,255],y:[0,1]
    NNE_INPUT_NORM_U8  = 0x2        //y=(x-0)/1;         //x:[0,255],y:[0,255]

}NNE_INPUT_NORM_E;
```

【成员】

成员名称	描述
NNE_INPUT_NORM_ABS1	y=(x-128)/128; x:[0,255], y:[-1,1]
NNE_INPUT_NORM_01	y=(x-0)/255; x:[0,255], y:[0,1]
NNE_INPUT_NORM_U8	y=(x-0)/1; x:[0,255], y:[0,255]

【注意】

模型转换过程中的配置文件中需填写。

2.4.3 NNE_TASK_E

【说明】

算法所属任务类别。

【定义】

```
typedef enum NNE_TASK_E
{
    NNE_TASK_Classification  = 0x0,
    NNE_TASK_Detection  = 0x1,
    NNE_TASK_OTHER_TASK  = 0x2

}NNE_TASK_E;
```

【成员】

成员名称	描述
NNE_TASK_Classification	分类
NNE_TASK_Detection	检测
NNE_TASK_OTHER_TASK	其他

【注意】

目前没有使用该枚举值。

2.4.4 NNE_QUAN_METHOD_E

【说明】

FeatureMap 的量化方式。

Anyka Confidential For Only

【定义】

```
typedef enum NNE_QUAN_METHOD_E
{
    NNE_QUAN_METHOD_Hist_Symmetry  = 0x0,
    NNE_QUAN_METHOD_USR_SET  = 0x1

}NNE_QUAN_METHOD_E;
```

【成员】

成员名称	描述
NNE_QUAN_METHOD_Hist_Symmetry	对称量化，直方图统计数据分布。
NNE_QUAN_METHOD_USR_SET	用户设置每一层的量化系数。

【注意】

使用方法参考《AnyCloud39AV100 NNE 网络移植编程手册》3.8.1 节，量化方法介绍。

2.4.5 NNE_XRELU_TYPE_E

【说明】

ReLU 系列激活函数类别。

【定义】

```
typedef enum NNE_XRELU_TYPE_E
{
    NNE_XRELU_TYPE_No_Relu  = 0x0,
    NNE_XRELU_TYPE_Relu  = 0x1,
    NNE_XRELU_TYPE_PRelu  = 0x2,
    NNE_XRELU_TYPE_LRelu  = 0x3,
    NNE_XRELU_TYPE_Relu4  = 0x4,
    NNE_XRELU_TYPE_Relu6  = 0x5,
    NNE_XRELU_TYPE_Relu8  = 0x6

}NNE_XRELU_TYPE_E;
```

【成员】

成员名称	描述
NNE_XRELU_TYPE_No_Relu	无激活函数
NNE_XRELU_TYPE_Relu	ReLU
NNE_XRELU_TYPE_PRelu	PReLU
NNE_XRELU_TYPE_LRelu	LeakyReLU
NNE_XRELU_TYPE_Relu4	Clip,min=0,max=4
NNE_XRELU_TYPE_Relu6	Clip,min=0,max=6
NNE_XRELU_TYPE_Relu8	Clip,min=0,max=8

【注意】

无

2.4.6 NNE_OP_TYPE_E

【说明】

定点模型文件中的算子类别。

【定义】

```
typedef enum NNE_OP_TYPE_E
{
    NNE_OP_TYPE_Conv = 0x0,
    NNE_OP_TYPE_Input = 0x1,
    NNE_OP_TYPE_Global_Pooling = 0x2,
    NNE_OP_TYPE_Other_Pooling = 0x3,
    NNE_OP_TYPE_Flatten = 0x4,
    NNE_OP_TYPE_Concat_Shuffle_Split = 0x5,
    NNE_OP_TYPE_Concat_Shuffle = 0x6,
    NNE_OP_TYPE_Concat_C = 0x7,
    NNE_OP_TYPE_Resize_W = 0x8,
    NNE_OP_TYPE_Resize_H = 0x9,
    NNE_OP_TYPE_Reshape = 0xa,
    NNE_OP_TYPE_Softmax = 0xb,
    NNE_OP_TYPE_Padding = 0xc,
    NNE_OP_TYPE_Add = 0xd,
    NNE_OP_TYPE_Build_Concat_Group = 0xe,
    NNE_OP_TYPE_Slice = 0xf,
    NNE_OP_TYPE_Split_C = 0x10,
    NNE_OP_TYPE_Split_H = 0x11,
    NNE_OP_TYPE_Split_W = 0x12,
    NNE_OP_TYPE_Concat_H = 0x13,
    NNE_OP_TYPE_Concat_W = 0x14,
    NNE_OP_TYPE_Transpose = 0x15,
    NNE_OP_TYPE_Split_W_CPU = 0x16,
    NNE_OP_TYPE_Concat_W_CPU = 0x17

}NNE_OP_TYPE_E;
```

【成员】

成员名称	描述
NNE_OP_TYPE_Conv	卷积层
NNE_OP_TYPE_Input	输入层
NNE_OP_TYPE_Global_Pooling	全局池化
NNE_OP_TYPE_Other_Pooling	其他池化
NNE_OP_TYPE_Flatten	Flatten
NNE_OP_TYPE_Concat_Shuffle_Split	Concat+Shuffle+Split 的组合

成员名称	描述
NNE_OP_TYPE_Concat_Shuffle	Concat+Shuffle 的组合
NNE_OP_TYPE_Concat_C	channel 维度的拼接
NNE_OP_TYPE_Resize_W	Width 维度的上采样，scale=2，nearest。
NNE_OP_TYPE_Resize_H	Height 维度的上采样，scale=2，nearest。
NNE_OP_TYPE_Reshape	Reshape
NNE_OP_TYPE_Softmax	Softmax
NNE_OP_TYPE_Padding	Padding
NNE_OP_TYPE_Add	按元素加法
NNE_OP_TYPE_Build_Concat_Group	算子转换过程中间算子
NNE_OP_TYPE_Slice	Slice
NNE_OP_TYPE_Split_C	Channel 维度的 Split
NNE_OP_TYPE_Split_H	Height 维度的 Split
NNE_OP_TYPE_Split_W	Width 维度的 Split
NNE_OP_TYPE_Concat_H	Height 维度的 Concat
NNE_OP_TYPE_Concat_W	Width 维度的 Concat
NNE_OP_TYPE_Transpose	Transpose
NNE_OP_TYPE_Split_W_CPU	Width 维度的 Split，使用 CPU 计算
NNE_OP_TYPE_Concat_W_CPU	Width 维度的 Concat，使用 CPU 计算

【注意】

Reshape 如果出现在网络最后会删除并提示用户在后处理中实现，否则会报错不支持。定点模型中的算子是在 onnx 算子基础上经过合并、拆分等优化后的结果。

2.4.7 NNE_POOL_TYPE_E

【说明】

池化类型。

【定义】

```
typedef enum NNE_POOL_TYPE_E
{
    NNE_POOL_TYPE_MAX  = 0x0,
    NNE_POOL_TYPE_AVG  = 0x1

}NNE_POOL_TYPE_E;
```

【成员】

成员名称	描述
NNE_POOL_TYPE_MAX	最大值池化
NNE_POOL_TYPE_AVG	均值池化

【注意】

无

Anyka Confidential For Giec Use Only

2.4.8 NNE_DATA_TYPE_E

【说明】

数据类型。

【定义】

```
typedef enum NNE_DATA_TYPE_E
{
    NNE_DATA_TYPE_U8  = 0x0,
    NNE_DATA_TYPE_S8  = 0x1,
    NNE_DATA_TYPE_F32  = 0x2,
    NNE_DATA_TYPE_U32  = 0x3,
    NNE_DATA_TYPE_S32  = 0x4

}NNE_DATA_TYPE_E;
```

【成员】

成员名称	描述
NNE_DATA_TYPE_U8	unsigned char
NNE_DATA_TYPE_S8	signed char
NNE_DATA_TYPE_F32	float32
NNE_DATA_TYPE_U32	unsigned int
NNE_DATA_TYPE_S32	signed int

【注意】

NNE_V1 乘加器仅支持 signed char 类型。网络输入可特殊处理，支持 signed char 和 unsigned char 类型。

2.4.9 NNE_DATA_ARRANGE_E

【说明】

数据块“C,H,W”三个维度数据的排序方式。

【定义】

```
typedef enum NNE_DATA_ARRANGE_E
{
    NNE_DATA_ARRANGE_HWC  = 0x0,
    NNE_DATA_ARRANGE_CHW  = 0x1,
    NNE_DATA_ARRANGE_HCW  = 0x2

}NNE_DATA_ARRANGE_E;
```

【成员】

成员名称	描述
NNE_DATA_ARRANGE_HWC	先排 C 维度，再排 W 维度，再排 H 维度。
NNE_DATA_ARRANGE_CHW	先排 W 维度，再排 H 维度，再排 C 维度。

Anyka Confidential For Giec Use Only

成员名称	描述
NNE_DATA_ARRANGE_HCW	先排 W 维度，再排 C 维度，再排 H 维度。

【注意】

NNE_DATA_ARRANGE_HWC 的排序方式同 opencv 的 imread 函数读入后的排序方式；NNE_DATA_ARRANGE_CHW 的排序方式同 pytorch 推理过程中[N,C,H,W]的排序方式（其中 N==1）；NNE_DATA_ARRANGE_HCW 的排序方式是 NNE_V1 内部排序方式。

2.4.10 NNE_MODEL_HEADER

【说明】

定点模型文件的文件头。

【定义】

```
typedef struct NNE_MODEL_HEADER        //12*(32bit)
{
    T_U32 model_parse_version;
    T_U32 header_size;
    T_U32 network_size;
    T_U32 param_size;
    T_U32 model_size;
    T_U32 model_label;
    T_U32 v_abcd;
    T_U32 reserved1;
    T_U32 reserved2;
    T_U32 reserved3;
    T_U32 reserved4;
    T_U32 reserved5;

}NNE_MODEL_HEADER;
```

【成员】

成员名称	描述
model_parse_version	模型文件解析方式版本号
header_size	模型文件头的 byte 数量
network_size	描述网络结构信息的 byte 数量
param_size	描述参数信息的 byte 数量
model_size	整个模型文件的 byte 数量
model_label	用于模型版本号管理，区分模型类别
v_abcd	用于模型版本号管理，可提取出 Va.b.c

【注意】

model_label 和 v_abcd 的数据来自于 PC 端工具使用到的配置文件（main_cfg_xxx.txt）。

2.4.11 NNE_INPUT_OUTPUT_BLOB_INFO_S

【说明】

网络输入层/输出层单个数据块信息结构体。

【定义】

```
typedef struct NNE_INPUT_OUTPUT_BLOB_INFO_S
{
    T_VOID *VirAddr;
    T_Addr PhyAddr;
    T_U32 size;
    T_U32 w;
    T_U32 h;
    T_U32 c;
    T_U32 scale;
    T_U32 data_type;      //T_U8/T_S8/...
    T_U32 data_arrange_type;
    T_U32 memory_type;   //[0:dma_memory,will not copy] [1:normal_memory,will copy]

}NNE_INPUT_OUTPUT_BLOB_INFO_S;
```

【成员】

成员名称	描述
VirAddr	存储 Blob 数据的虚拟地址
PhyAddr	存储 Blob 数据的物理地址
size	Blob 数据量
w	Blob 数据维度中的 width
h	Blob 数据维度中的 height
c	Blob 数据维度中的 channle
scale	Blob 数据相对于真实值的缩放倍数
data_type	Blob 数据类型，如 T_S8/T_U8。
data_arrange_type	Blob 数据在内存中的排列方式。
memory_type	输入数据是否拷贝。

【注意】

当前 data_type、data_arrange_type 不需要设置。

如果输入数据保存在 DMA 内存中，且在网络推理过程中不会改变，则 memory_type 设置为 0，内部不做拷贝，只做指针赋值；如果输入数据保存在普通内存中，或无法保证在网络推理过程中不会改变，则 memcpy_type 设置为 1，内部会将输入数据拷贝一份。

2.4.12 NNE_BLOB_VECTOR_S

【说明】

定义网络输入层/输出层数据块向量结构体。

【定义】

Anyka Confidential Giec Use Only

```
#define NNE_MAX_BLOB_CNT      12
typedef struct NNE_BLOB_VECTOR_S
{
    T_U32 blob_cnt;
    NNE_INPUT_OUTPUT_BLOB_INFO_S  array_blobs[NNE_MAX_BLOB_CNT];

}NNE_BLOB_VECTOR_S;
```

【成员】

成员名称	描述
blob_cnt	Blob 个数
array_blobs	Blob 信息结构体数组

【注意】

NNE_MAX_BLOB_CNT 为输入层/输出层数据块个数的最大值，当前值为 12。

2.4.13 NNE_POINT_S

【说明】

点的定义。

【定义】

```
typedef struct NNE_POINT_S
{
    T_S32 x;
    T_S32 y;

}NNE_POINT_S;
```

【成员】

成员名称	描述
x	row 方向坐标。
y	col 方向坐标。

【注意】

无

2.4.14 NNE_RECT_S

【说明】

矩形的定义。

【定义】

```
typedef struct NNE_RECT_S
{
    NNE_POINT_S ul;
```

Anyka Confidential For Giec only

```
    NNE_POINT_S br;

}NNE_RECT_S;
```

【成员】

成员名称	描述
ul	左上角（up_left）点坐标
br	右下角（bottom_right）点坐标

【注意】

无

2.4.15 NNE_Init

【功能】

初始化一个全局 handle，包含全局互斥锁等。

【语法】

```
T_S32 NNE_Init(const T_AKCNNLIB_COMMON_CB  *init_cb_fun);
```

【参数】

参数名称	描述	取值范围	输入/输出
init_cb_fun	回调函数结构体指针	-	输入

【返回值】

返回值	描述
0	成功。
非 0	失败。

【需求】

头文件：ak_nne_common.h

库文件：libak_nne.so 或者 libak_nne.a

【注意】

该函数由 ak_npu_init 调用，已封装在 libplat_npu.so 中，请用户不要调用。

2.4.16 NNE_DeInit

【功能】

释放全局 handle。

【语法】

```
T_S32 NNE_DeInit(T_VOID);
```

【参数】

参数名称	描述	取值范围	输入/输出

Anyka Confidential For Giec Use Only

void	-	-	-

【返回值】

返回值	描述
0	成功。
非 0	失败。

【需求】

头文件：ak_nne_common.h

库文件：libak_nne.so 或者 libak_nne.a

【注意】

如该函数由 ak_npu_deinit 调用，已封装在 libplat_npu.so 中，请用户不要调用。

2.4.17 NNE_Get_CallBackFun

【功能】

获取全局 handle 中的回调函数指针。

【语法】

```
T_VOID *NNE_Get_CallBackFun(T_VOID);
```

【参数】

参数名称	描述	取值范围	输入/输出
void	-	-	-

【返回值】

返回值	描述
回调函数指针	成功。
NULL	失败。

【需求】

头文件：ak_nne_common.h

库文件：libak_nne.so 或者 libak_nne.a

【举例】

```
T_AKCNNLIB_COMMON_CB  *cb_fun = NULL;
cb_fun = (T_AKCNNLIB_COMMON_CB  *)NNE_Get_CallBackFun();
```

【注意】

如果返回值为 NULL，请检查是否已经调用 ak_npu_init 函数。

2.4.18 NNE_Create_CH

【功能】

初始化指定网络句柄。

Anyka Confidential For Giec Use Only

【语法】

```
T_S32 NNE_Create_CH(T_VOID *model, T_S32 size, T_VOID **nne_net_handle, T_U32 *channel_id);
```

【参数】

参数名称	描述	取值范围	输入/输出
model	指向“模型文件路径”或“存储模型的buffer”。	-	输入
size	如果传入“模型文件路径”，则size填0；如果传入“存储模型的buffer”则size表示 buffer 大小。	>=0	输入
nne_net_handle	网络句柄	-	输出
channel_id	网络句柄 id。	>=0	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【需求】

头文件：ak_nne_common.h

库文件：libak_nne.so 或者 libak_nne.a

【举例】

```
const char *model_file = “/mnt/mobilenet_v1.bin”;
T_S32 size = 0;
T_VOID *net_handle = NULL;
T_U32 channel_id;
ret = NNE_Create_CH((T_VOID *)model_file,    size,    &net_handle, &channel_id);
if (ret != 0)
{
    cb_fun->ak_FunPrintf((T_pCSTR)("NNE_Create_CH error\n"));
    return ret;
}
```

【注意】

创建网络句柄的方式有 2 种，第一种是传入模型文件路径，size 填写 0，在函数内会读取该模型文件；第二种是在 NNE_Create_CH 函数外已将模型文件读入 buffer 中，此处传入 buffer 指针和 buffer 的 size。

与 NNE_Destroy_CH()配对使用。

2.4.19 NNE_Destroy_CH

【功能】

销毁指定网络句柄。

Confidential For Only

【语法】

```
T_S32 NNE_Destroy_CH(T_VOID *nne_net_handle);
```

【参数】

参数名称	描述	取值范围	输入/输出
nne_net_handle	网络句柄	-	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【需求】

头文件：ak_nne_common.h
库文件：libak_nne.so 或者 libak_nne.a

【举例】

```
ret = NNE_Destroy_CH(net_handle);
if (ret != 0)
{
    cb_fun->ak_FunPrintf((T_pCSTR)("NNE_Destroy_CH error\n"));
    return ret;
}
```

【注意】

与 NNE_Create_CH()配对使用。

2.4.20 NNE_Run

【功能】

执行指定网络正向推理。与 NNE_Run_Interrupt 功能相同。

【语法】

```
T_S32 NNE_Run(T_VOID *nne_net_handle, NNE_BLOB_VECTOR_S *input_blobs,
NNE_BLOB_VECTOR_S *output_blobs);
```

【参数】

参数名称	描述	取值范围	输入/输出
nne_net_handle	网络句柄	-	输入
input_blobs	输入数据向量	-	输入
output_blobs	输出数据向量	-	输出

Anyka Confidential For Giec Use Only

【返回值】

返回值	描述
0	成功。
非 0	失败。

【需求】

头文件：ak_nne_common.h

库文件：libak_nne.so 或者 libak_nne.a

【举例】

```
//input
NNE_BLOB_VECTOR_S  input_blob_vectors;
input_blob_vectors.blob_cnt = 1;
input_blob_vectors.array_blobs[0].w = usr_cfg.input_w;
input_blob_vectors.array_blobs[0].h = usr_cfg.input_h;
input_blob_vectors.array_blobs[0].c = usr_cfg.input_c;
input_blob_vectors.array_blobs[0].scale = 1;
input_blob_vectors.array_blobs[0].VirAddr  = (T_VOID*)pFrame->pData;
input_blob_vectors.array_blobs[0].PhyAddr = (T_Addr)pFrame->pa;
input_blob_vectors.array_blobs[0].size = pFrame->size;

//output
NNE_BLOB_VECTOR_S  output_blob_vectors;

ret = NNE_Run(net_handle, &input_blob_vectors, &output_blob_vectors);
if (ret != 0)
{
    cb_fun->ak_FunPrintf((T_pCSTR)("\nNNE_Run Error.\n"));
}

//result
//此处结构体(Param_Intput_S、Param_Output_S)及后处理函数(nne_post_process)由用户自定义。
int  nne_post_process(NNE_BLOB_VECTOR_S  *input_blobs, NNE_BLOB_VECTOR_S *output_blobs, Param_Intput_S input_param, Param_Output_S *output_param);
```

【注意】

完整网络推理过程被分成三部分，预处理、主网络推理和后处理。此处 src_blobs 是主网络推理的输入，dst_blobs 是主网络推理的输出，并非最终输出，后续还需后处理才能得到网络最终的输出结果。

NNE_Run 和 NNE_Run_Interrupt 功能相同，一般情况下，调用 NNE_Run_Interrupt 耗时更短，请用户优先使用 NNE_Run_Interrupt，如果 SDK 不支持“中断模式”时使用 NNE_Run。使用 NNE_Run_Interrupt 需要注意检查环境中是否已加载“ak_npu.ko”。

2.4.21 NNE_Run_Interrupt

【功能】

执行指定网络正向推理。与 NNE_Run 功能相同。

【语法】

```
T_S32 NNE_Run_Interrupt(T_VOID *nne_net_handle, NNE_BLOB_VECTOR_S *input_blobs, NNE_BLOB_VECTOR_S *output_blobs);
```

【参数】

参数名称	描述	取值范围	输入/输出
nne_net_handle	网络句柄	-	输入
input_blobs	输入数据向量	-	输入
output_blobs	输出数据向量	-	输出

【返回值】

返回值	描述
0	成功。
非 0	失败。

【需求】

头文件：ak_nne_common.h

库文件：libak_nne.so 或者 libak_nne.a

【举例】

```
//input
NNE_BLOB_VECTOR_S  input_blob_vectors;
input_blob_vectors.blob_cnt = 1;
input_blob_vectors.array_blobs[0].w = usr_cfg.input_w;
input_blob_vectors.array_blobs[0].h = usr_cfg.input_h;
input_blob_vectors.array_blobs[0].c = usr_cfg.input_c;
input_blob_vectors.array_blobs[0].scale = 1;
input_blob_vectors.array_blobs[0].VirAddr = (T_VOID*)pFrame->pData;
input_blob_vectors.array_blobs[0].PhyAddr = (T_Addr)pFrame->pa;
input_blob_vectors.array_blobs[0].size = pFrame->size;

//output
NNE_BLOB_VECTOR_S  output_blob_vectors;

ret = NNE_Run_Interrupt(net_handle, &input_blob_vectors, &output_blob_vectors);
if (ret != 0)
{
    cb_fun->ak_FunPrintf((T_pCSTR)("\nNNE_Run Error.\n"));
}
```

Confidential For Only

```
//result
//此处结构体(Param_Intput_S、Param_Output_S)及后处理函数(nne_post_process)由用户自定义。
int nne_post_process(NNE_BLOB_VECTOR_S *input_blobs, NNE_BLOB_VECTOR_S *output_blobs, Param_Intput_S input_param, Param_Output_S *output_param);
```

【注意】

完整网络推理过程被分成三部分，预处理、主网络推理和后处理。此处 src_blobs 是主网络推理的输入，dst_blobs 是主网络推理的输出，并非最终输出，后续还需后处理才能得到网络最终的输出结果。

NNE_Run 和 NNE_Run_Interrupt 功能相同，一般情况下，调用 NNE_Run_Interrupt 耗时更短，请用户优先使用 NNE_Run_Interrupt，如果 SDK 不支持“中断模式”时使用 NNE_Run。使用 NNE_Run_Interrupt 需要注意检查环境中是否已加载“ak_npu.ko”。

2.4.22 NNE_Get_Lib_Version

【功能】

获取 NNE 库版本号。

【语法】

```
T_pCSTR NNE_Get_Lib_Version(T_VOID);
```

【参数】

参数名称	描述	取值范围	输入/输出
void	-	-	-

【返回值】

返回值	描述
字符串指针	NNE 库版本号。

【需求】

头文件：ak_nne_common.h

库文件：libak_nne.so 或者 libak_nne.a

【举例】

```
cb_fun->ak_FunPrintf((T_pCSTR)("\n%s\n"), NNE_Get_Lib_Version );
```

【注意】

无

2.5 错误码

错误代码	宏定义	描述
0	E_NNE_ERR_RIGHT	正确
-1	E_NNE_ERR_OTHER	其他错误

Anyka Confidential For Giec Use Only

错误代码	宏定义	描述
-2	E_NNE_ERR_NOMEM	申请普通内存失败
-3	E_NNE_ERR_NODMAMEM	申请 DMA 内存失败
-4	E_NNE_ERR_FILE_OPERATION	文件打开/读/写错误
-5	E_NNE_ERR_CFG_INFO	配置文件错误
-6	E_NNE_ERR_NULL_PTR	函数参数中有空指针
-7	E_NNE_ERR_CHN_EXIST	通道已经存在，指针非空
-8	E_NNE_ERR_INFO_INCORRECT	信息冲突
-9	E_NNE_ERR_ILLEGAL_PARAM	参数范围不满足要求
-10	E_NNE_ERR_NOT_SUPPORT_OP	不支持的算子

3 调用流程

ak_npu_init 和 ak_npu_deinit 完成 NNE 硬件资源的初始化和释放，如果有多个用户使用 NNE 资源，只需调用一次。如图 3-1 所示。

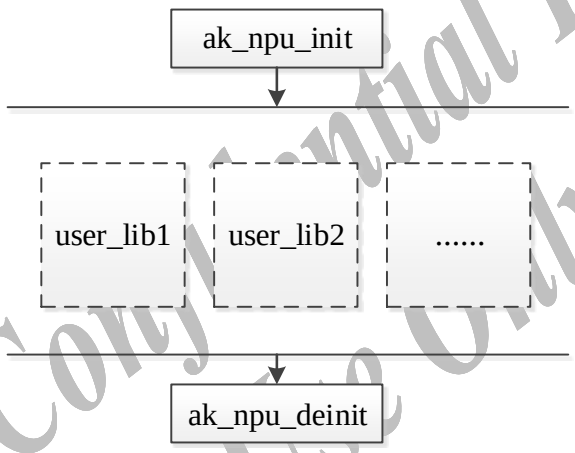

图 3-1 ak_npu_init 和 ak_npu_deinit 的调用流程

ak_npu_get_version 函数可在任意时候调用。

ak_npu_reset_hw 用于重置 NNE 模块，当 NNE 模块出现错误时调用。

用户自定义库可直接调用 libak_nne 接口，也可以基于 libak_nne 封装。用户自定义库内部可包含一个或多个网络。libak_nne 主要接口函数调用流程如图 3-2 所示。

Anyka Confidential For Cisc Use Only

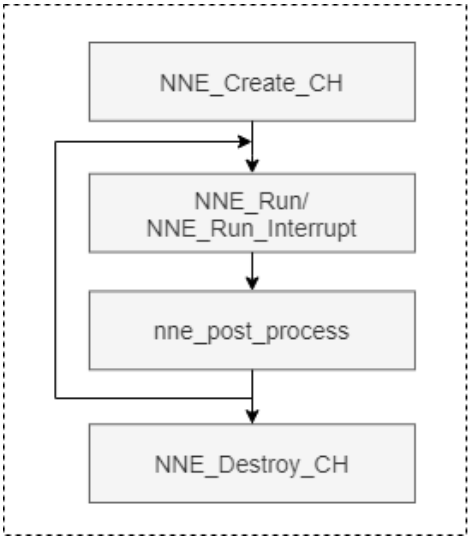

图 3-2 libak_nne 库主要接口调用流程

其中 nne_post_process 不是 libak_nne 库接口函数，而是用户自定义函数，用于实现网络后处理。

NNE_Run 和 NNE_Run_Interrupt 功能相同，优先使用 NNE_Run_Interrupt，如果 SDK 版本不支持则使用 NNE_Run。

ak_nne_common.h 中的 NNE_Init 和 NNE_DeInit 两个函数，请用户不要调用。ak_npu_init 函数内已调用 NNE_Init 及其他必要函数；ak_npu_deinit 函数内已调用 NNE_DeInit 及其他必要函数。

NNE_Get_CallBackFun 函数在 ak_npu_init 和 ak_npu_deinit 之间调用，返回指针，无需释放。

NNE_Get_Model_Header 函数在 NNE_Create_CH 和 NNE_Destroy_CH 之间调用。

NNE_Get_Lib_Version 可在程序任意时间调用。

Anyka Confidential For Giec Use Only